

Neural-inspired Advances in Integral Cryptanalysis

Liu Zhang^{1,3}, Yiran Yao³, Danping Shi^{2,4,5}, Dongchen Chai¹, Jian Guo³, and Zilong Wang¹

¹ School of Cryptology, Xidian University, Xi'an, China

liu.zhang@ntu.edu.sg, chaidc@foxmail.com, zlwang@xidian.edu.cn

² State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China shidanping@iie.ac.cn

³ School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore yiran005@e.ntu.edu.sg, guojian@ntu.edu.sg

⁴ State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, China

⁵ School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

Abstract. The studies by Gohr et al. at Crypto 2019 and subsequent related works have demonstrated that neural networks can offer new insights for cryptanalysis. Building on this insight, we leverage neural networks to learn features closely associated with integral properties and use neural network-derived distinguishers as a benchmark for the automatic search. This approach not only confirms the value of deep learning in feature discovery but also shows that neural-guided insights can improve the performance of classical cryptanalytic methods.

Neural networks motivate the development of a more effective framework for identifying integral distinguishers. Comparative results show that existing automated methods, limited by search efficiency and the large search space, often fail to locate optimal distinguishers. In contrast, neural networks can directly identify high-quality integral distinguishers and reveal associated non-random structural features that are typically missed by traditional approaches. Based on these neural-network results, we refine the meet-in-the-middle search framework, thereby improving the trade-off between accuracy and computational cost. Notably, under the assumption of full-state key XOR with independent round keys, the refined framework achieves the known theoretical upper bound for key-independent integral distinguishers on non-standard SKINNY.

Integral distinguishers identified by neural networks, whose underlying non-random features are successfully interpreted through Boolean function analysis, are translated into classical forms, enhancing integral key-recovery attacks. In particular, we propose a 16-round key-recovery attack on SKINNY- n - n based on a general integral key-recovery approach, improving the best-known result by two rounds under the single-tweakey setting. Furthermore, we present key-recovery attacks on 18-round SKINNY- n - $2n$ and 20-round SKINNY- n - $3n$ using integral distinguishers under the single-tweakey setting for the first time.

¹ Liu Zhang and Yiran Yao are the main contributors to this work.

Keywords: Neural Network · Feature Extraction · Classical Interpretation · Integral Property · SKINNY

1 Introduction

Integral cryptanalysis, first introduced by Knudsen in [16], analyzes the algebraic structure of a block cipher by computing the sum of ciphertexts derived from a set of plaintexts that span a linear subspace. It tracks how integral properties (ALL, BALANCE, CONSTANT, and UNKNOWN) propagate through the internal state words via the cipher’s round transformations. The division property, introduced in [19], provides a general and precise framework for identifying integral distinguishers by describing a set $\mathbb{X} \subseteq \mathbb{F}_2^n$ using a subset of indicator vectors $u \in \mathbb{F}_2^n$. These vectors are classified into two categories: the 0 (balance)-subset and the *unknown*-subset. To further improve granularity, the bit-based division property was extended in [21] to a three-subset form by adding the 1-subset. This extension builds naturally upon the two-subset approach, allowing more detailed tracking of property propagation. Also in 2016, Boura and Canteaut introduced the notion of parity sets [5], offering an alternative view of the division property by connecting it with the algebraic normal form of the cipher’s components. Lambin et al. extended the analysis by considering ciphers with linearly equivalent S-boxes and successfully identified high-round integral distinguishers for the RECTANGLE [17]. More recently, at FSE 2023, Beyne and Verbauwhede introduced the notion of generalized integral properties, which extend traditional integral distinguishers by considering not only mapping-based properties on ciphertexts but also transformations applied to the plaintext inputs [4]. While this generalization enables the discovery of more powerful distinguishers, it also significantly expands the search space, making automated exploration substantially more complex.

One of the main challenges in identifying integral distinguishers lies in effectively modeling how integral properties propagate through a cipher’s round functions. Early works evaluated this propagation using breadth-first search algorithms [19,21,20], but such methods quickly become computationally infeasible for block ciphers with large state sizes. To address this limitation, Xiang et al. proposed a more efficient MILP-based automatic search technique [23], utilizing the division property with two subsets. This approach was further enhanced by Wang et al., who introduced a more precise MILP model using a three-subset division property along with pruning strategies and faster propagation mechanisms [22]. Hu et al. developed the monomial prediction technique, which accelerates monomial enumeration and enables the recovery of more superpolies [15]. Lambin et al. incorporated linear mappings into the analysis, enabling the discovery of high-round integral distinguishers for RECTANGLE [17]. Most recently, Beyne and Verbauwhede proposed using algebraic transition matrices to search for generalized integral properties that incorporate both input and output transformations. Their approach was successfully applied to PRESENT [4].

Our contributions. Artificial intelligence-based approaches to symmetric cryptanalysis, such as those presented at CRYPTO 2019 [10], EUROCRYPT 2021 [3], and ASIACRYPT 2023 [1] have demonstrated that neural networks can extract additional non-random features even from limited data. In the context of integral cryptanalysis, this work demonstrates that neural networks can reveal distinguishing features that existing automated search models often miss. These insights are then formalized and integrated into refined search frameworks, leading to the discovery of stronger distinguishers and more effective cryptanalytic attacks. The main contributions are as follows:

- **Neural Network-Based Feature Discovery and Boolean Interpretation.** We introduce parity sets to preprocess ciphertext sets, guiding neural networks to learn features closely tied to integral properties. By addressing the interpretability challenge through Boolean functions, neural networks are shown to reveal more effective integral distinguishers, capturing structural characteristics often missed by classical techniques. For example, in SKINNY-64-64, automated methods need 15 active bits to identify an 8-round distinguisher, while neural networks achieve the same with only 8. Beyond this, neural networks, as statistical learners, can detect key-dependent integral properties that lead to weak-key distinguishers. Their ability to efficiently extract and represent such features, when paired with Boolean interpretation, makes them a practical and powerful tool in modern cryptanalysis.
- **Refining Automated Search Using Neural Guidance.** A comparison between neural networks and existing automated models shows that current automated search techniques have not yet achieved optimal performance. Neural networks often identify key features by linearly combining ciphertext bits, with the number of possible combinations theoretically up to 2^n (where n denotes the block size), making full-space exploration impractical for traditional methods. However, neural networks can effectively prune the candidate space, enabling automated approaches to avoid exhaustive search. Building on this insight, we refine a meet-in-the-middle automated search framework to better balance precision and complexity. The improved model identifies an 11-round key-independent and a 12-round key-dependent integral distinguisher for SKINNY-64-64 with 60 active bits, as shown in Table 1². Under the assumptions of round-key independence and full-state key XOR, the refined framework also reaches the theoretical upper bound for key-independent integral distinguishers on non-standard SKINNY, requiring only 60 active bits.
- **Enhanced Key-Recovery Attacks.** The features discovered by neural networks, once interpreted through Boolean function analysis, can be transformed into classical integral distinguishers and further extended to support key-recovery attacks. Notably, a 7-round integral distinguisher with a single active plaintext cell, based on a linear combination of ciphertext

² Our implementation is available at <https://github.com/CryptAnalystDesigner/skinny-automatic-and-AI-analysis>

bits, is derived from features learned by neural networks and interpreted through Boolean function analysis. This distinguisher enables a 16-round key-recovery attack on SKINNY-n-n, utilizing 4 rounds of forward decryption and 5 rounds of backward encryption—surpassing the previous best-known result by two rounds in the single-tweakey setting. A summary of these results is presented in Table 2.

Table 1. The integral distinguisher on SKINNY

Cipher	Round	Type	Data	Key Space	Reference
SKINNY-64-64	11*	Key-Independent	2^{63}	2^{64}	[7]
	11	Key-Independent	2^{60}	2^{64}	Sect.5.2
	11	Key-Dependent	2^{60}	2^{62}	Sect.6.2
	12	Key-Dependent	2^{60}	2^{62}	Sect.6.1
SKINNY-128-128	11	Key-Independent	2^{120}	2^{128}	Sect.5.2

*: After the 11th application of the SubCells operation

Table 2. The integral attack on SKINNY in single tweakey setting

Cipher	Round	Configure	Time	Data	Reference
SKINNY-64-64	14	4+6+4	$2^{48.087}$	2^{48}	[2]
	16	4+7+5	2^{50}	2^{50}	Sect.4.4
SKINNY-64-128	18	4+7+7	2^{114}	2^{50}	Sect.4.4
SKINNY-64-192	20	4+7+9	2^{178}	2^{50}	Sect.4.4
SKINNY-128-128	14	4+6+4	$2^{96.006}$	2^{96}	[2]
	16	4+7+5	2^{98}	2^{98}	Sect.4.4
SKINNY-128-256	18	4+7+7	2^{226}	2^{98}	Sect.4.4
SKINNY-128-384	20	4+7+9	2^{354}	2^{98}	Sect.4.4

The remainder of the paper is organized as follows. Section 2 provides the necessary background and preliminaries. Section 3 outlines the overall motivation and core methodology. In Section 4, short-round integral distinguishers are constructed using neural networks and Boolean functions, and an improved key recovery attack is developed. Section 5 introduces a refined model to capture better the features learned by the network. Section 6 presents key-dependent integral distinguishers. Finally, Section 7 concludes the paper.

2 Preliminaries

2.1 Brief Description of SKINNY

SKINNY is a family of tweakable block ciphers [2], where SKINNY- n operates on an n -bit block. Bits are indexed from left to right. Each state is structured as a 4×4 array of c -bit cells. The input state of the r th round, denoted $\mathbf{S}_r$, is indexed row-wise as $\mathbf{S}_r[0]$ to $\mathbf{S}_r[15]$, where $\mathbf{S}_r[4i+j]$ refers to the cell in row i , column j .

For each block size n , SKINNY- n supports three tweakkey sizes: $t = n$, $t = 2n$, and $t = 3n$. The notation SKINNY- n - t denotes the instance with block size n and tweakkey size t . The t -bit tweakkey is organized into t/n separate 4×4 arrays, denoted TK_z for $z \in \{1, \dots, t/n\}$. At the beginning of each round, each tweakkey undergoes a permutation. Furthermore, every cell in the first and second rows of TK_2 and TK_3 is updated individually using a linear feedback shift register, with the update rule specified in [2]. The round function itself consists of five operations: SubCells (SB), AddConstants (AC), AddRoundTweakey (AK), ShiftRows (SR), and MixColumns (MC), as illustrated in Fig. 1.

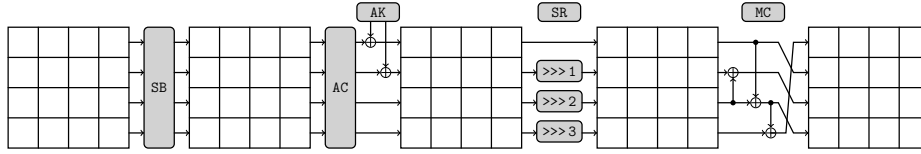


Fig. 1. The round function of SKINNY

SB substitutes each state cell using a c -bit S-box. For SKINNY-64 (resp. SKINNY-128), $c = 4$ (resp. $c = 8$). AC updates the state by XORing it with round-dependent constants. AK modifies the state by XORing the first two rows with the t/n tweakkey arrays. SR cyclically shifts the i -th row to the right by i cells. MC applies a linear diffusion layer by multiplying each column of the state with a fixed binary matrix

$$\begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

2.2 Integral Property

Notation. Let $\mathbb{F}_2$ denote the finite field $\{0, 1\}$, and let $a = (a_0, a_1, \dots, a_{m-1}) \in \mathbb{F}_2^m$ represent an m -bit vector, where $a[i]$ denotes the i -th bit of a . The Hamming weight $wt(a)$ is defined as $wt(a) = \sum_{i=0}^{m-1} a[i]$. For any $v \in \mathbb{F}_2^m$ and $v' \in \mathbb{F}_2^m$, we denote $v \succeq v'$ if $v_i \geq v'_i$ holds for all $i = 0, 1, \dots, m-1$.

Integral cryptanalysis considers a set of chosen plaintexts in which some bits take all possible values while the remaining bits are fixed. After encryption, the XOR of all resulting ciphertexts is evaluated. If this XOR is always zero, the cipher is said to admit an integral distinguisher. The division property offers a more precise and general framework for identifying such distinguishers.

Bit Product Functions π_u [19]. Let $\pi_u : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ be a function for any $u \in \mathbb{F}_2^m$. Let $x \in \mathbb{F}_2^m$ be an input of π_u , and $\pi_u(x)$ is the AND of $x[i]$ satisfying $u[i] = 1$, namely, it is defined as

$$\pi_u(x) := \prod_{i=0}^{m-1} x[i]^{u[i]}.$$

Definition 1 (Algebraic Normal Form). The Algebraic Normal Form (ANF) of a Boolean function $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ can be expressed as

$$f(x) = f(x_0, \dots, x_{m-1}) = \bigoplus_{u \in \mathbb{F}_2^m} a_u \pi_u(x),$$

where $a_u \in \mathbb{F}_2$ and $\pi_u(x)$ is called a monomial. If the coefficient of $\pi_u(x)$ in f is 1, we say $\pi_u(x)$ is contained in f , denoted by $\pi_u(x) \rightarrow f$.

Definition 2 (Bit-based Division Property using Two Subsets [21]). A set $\mathbb{X} \in \mathbb{F}_2^m$ has division property $D_{\mathbb{K}}^m$, where $\mathbb{K} \in \mathbb{F}_2^m$ is a set, if for all $u \in \mathbb{F}_2^m$, we have

$$\bigoplus_{x \in \mathbb{X}} \pi_u(x) = \begin{cases} \text{unknown} & \text{if there is } k \in \mathbb{K} \text{ s.t. } u \succeq k, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 3 (Monomial Prediction [15]). If there exists a monomial sequence that satisfies

$$\pi_{u(0)}(x^{(0)}) \rightarrow \pi_{u(1)}(x^{(1)}) \rightarrow \dots \rightarrow \pi_{u(r)}(x^{(r)}),$$

we call that there is a monomial trail connecting $\pi_{u(0)}(x^{(0)})$ and $\pi_{u(r)}(x^{(r)})$, denoted by $\pi_{u(0)}(x^{(0)}) \rightsquigarrow \pi_{u(r)}(x^{(r)})$. If the number of monomial trails connecting $\pi_{u(0)}(x^{(0)})$ and $\pi_{u(r)}(x^{(r)})$ is odd, then we have $\pi_{u(0)}(x^{(0)}) \rightarrow \pi_{u(r)}(x^{(r)})$.

Beyne and Verbauwheide [4] provide a unified summary of integral properties, encompassing the original integral property [16], the division property [19,21], and the property derived from linearly equivalent S-boxes [17], which was further extended by Derbez and Fouque [7].

Definition 4 (Integral Property [4]). Let $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a vectorial Boolean function. An integral property for F is a pair $(\mathbb{X}, g)$ with $\mathbb{X} \in \mathbb{F}_2^n$ and $g : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ is a mapping, and its evaluation is equal to

$$\sum_{x \in \mathbb{X}} g(F(x)).$$

3 Overview: From Neural Feature Extraction to Automated Integral Cryptanalysis

Deep learning can be used as an auxiliary tool to identify subtle non-random patterns in ciphertext that may be difficult to capture with existing automated methods. Neural networks have been employed to learn integral-related properties from ciphertext sets for ciphers such as PRESENT, RECTANGLE, LBLOCK, and SPECK32/64, achieving longer-round integral distinguishers than those identified by previous works when activating four plaintext bits [24].

SKINNY serves as an ideal case study for this work for two main reasons. First, its S-box has a weaker algebraic structure than AES, making it more vulnerable to integral cryptanalysis. Second, it benefits from a mature key-recovery framework that is explicitly built around integral distinguishers. These properties make SKINNY particularly well-suited for investigating how neural networks can be integrated with classical cryptanalytic methods.

Neural networks and automated search techniques can both serve as powerful tools in cryptanalysis, yet each has inherent limitations.

- *For Neural Network: Data Volume and Interpretability.* Neural networks excel at capturing subtle, high-dimensional statistical features directly from ciphertext data, without relying on extensive assumptions. Deep learning is fundamentally a statistical learning approach, meaning that all extracted features are derived from data. However, the high data complexity inherent to many attacks limits the practicality of using neural networks as a complete, standalone cryptanalysis tool. Moreover, neural networks are often treated as black-box models, making their interpretability a challenging problem. This stands in contrast to automated search techniques, which emphasize features that can be precisely characterized and explicitly explained.
- *For Automated Search Technique: What are Non-Random Features and Where do They Appear?* Differential, linear, and integral cryptanalysis first define and characterize the non-random features present in plaintext and ciphertext. Only with such definitions can automated search techniques be designed to locate the corresponding patterns. Without knowing what features to look for, no meaningful model can be built. And without knowing where those features might exist, automated methods are forced to search the entire space exhaustively. Automated search techniques need a compass to guide them toward better results.

A hybrid framework is proposed to combine neural networks with automated search techniques, leveraging their respective strengths while addressing their individual limitations. An overview of the framework is illustrated in Fig. 2.

- *Neural Feature Discovery And Interpretation (Section 4).* Neural networks are trained on parity-preprocessed ciphertext datasets to learn features indicative of integral properties. The resulting integral-neural distinguishers surpass those obtained by previous methods. Subsequently, the extracted

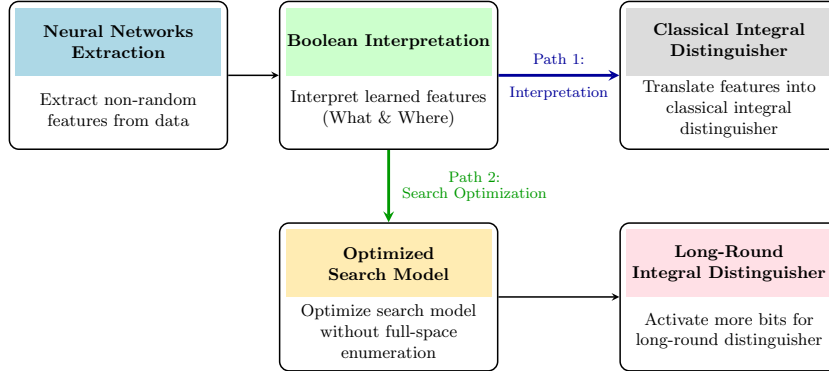


Fig. 2. Framework: from neural feature extraction to automated integral cryptanalysis

features are interpreted in the Conjunctive Normal Form of Boolean functions, revealing properties that are otherwise difficult to capture with existing automated search techniques.

- *Search Model Enhancement (Section 5, 6).* The results of the neural network suggest that existing automated search models do not fully capture all integral-related properties within ciphertexts. These features often appear as linear combinations of ciphertext bits, and the total number of possible combinations can be as large as 2^n , where n is the block size, posing a major challenge for tuning automated search models. However, Boolean function interpretation of the neural network provides explicit expressions of the underlying properties, enabling automated methods to avoid exhaustive full-space searches. Based on this insight, a refined meet-in-the-middle search framework is introduced to better balance precision and complexity. This improved model is then used to identify integral distinguishers for non-standard SKINNY, achieving the best-known theoretical upper bound.
- *Attack Construction (Sections 4.4).* The integral-neural distinguishers obtained through neural network training can be transformed into classical integral distinguishers via Boolean function interpretation, enabling key recovery attacks.

The results demonstrate that neural networks can serve as feature-discovery tools, uncovering cryptographic properties that conventional techniques overlook. The insights extracted by neural models can be interpreted, formalized, and integrated into enhanced classical methods, serving as a guide to direct classical techniques toward discovering stronger distinguishers and conducting more effective cryptanalytic attacks. This work demonstrates that combining neural networks with automated search techniques can improve performance and provides a basis for future hybrid frameworks in the field.

4 Neural Network-Based Feature Detection and Classical Interpretation

Cryptanalysis seeks to identify and exploit non-random patterns in plaintext and ciphertext to recover the secret key. Neural networks, known for their ability to extract hidden features from data, are well-suited as feature-extraction models in this context. While typically viewed as black boxes with limited interpretability, the binary nature of inputs and outputs in cryptographic tasks enables the internal decision processes of neural networks to be interpreted using Boolean functions. In this section, we combine neural networks with Boolean function analysis to construct lightweight, interpretable feature detectors that learn integral properties from ciphertext data, ultimately enabling the discovery of effective integral distinguishers and their subsequent use in key-recovery attacks.

4.1 New Data Formats: Retaining Only Integral-relevant Features

In artificial intelligence, data quality is critical. High-quality data improves the effectiveness of neural network training and enhances generalization performance. In contrast, low-quality or noisy data can degrade model accuracy and may cause the network to learn misleading or incorrect patterns.

When applying neural networks to cryptanalysis, it is crucial to ensure the *purity* of the features present in the data. Classical symmetric-key cryptanalysis includes techniques such as differential, linear, and integral cryptanalysis, each exploiting distinct statistical properties inherent in the data. As the objective of this work is to train neural networks to capture features specific to integral cryptanalysis, the data must be carefully curated to preserve only integral-related features, while minimizing or eliminating irrelevant patterns and noise.

Zahednejad et al. used ciphertext sets as inputs to neural networks, where positive samples were generated by encrypting partially enumerated plaintexts, and negative samples by encrypting random plaintexts [24]. However, such ciphertext sets may contain mixed features, including differential features. To ensure the data captures only integral-related features, we construct ciphertext sets using the parity-set method proposed in [5].

Definition 5 (Parity Set [5]). Let $\mathbb{X}$ be a set of elements in $\mathbb{F}_2^m$, the parity set of $\mathbb{X}$, denoted by $\mathcal{U}(\mathbb{X})$, is the subset of $\mathbb{F}_2^m$ defined by

$$\mathcal{U}(\mathbb{X}) = \{u \in \mathbb{F}_2^m : \bigoplus_{x \in \mathbb{X}} x^u = 1\}.$$

We adopt the concept of the parity set to create datasets tailored for learning integral-related features using neural networks. Since directly analyzing the division property of the entire cipher state is generally impractical, cryptanalysis is usually conducted at the cell level, with the state-level division property obtained by combining the properties of individual cells. Based on this, two types of data formats are defined:

Definition 6 (Division Sequence). Let $\mathbb{X}$ be a set of elements in $\mathbb{F}_2^c$ where c is the S-box size, the division sequence of $\mathbb{X}$, denoted by $\mathbb{DS}$, is a sequence defined by

$$\mathbb{DS} = [\bigoplus_{x \in \mathbb{X}} x^u, u \in \mathbb{F}_2^c].$$

Definition 7 (Vectorial Division Sequence). Let $\mathbb{X}_0, \mathbb{X}_1, \dots, \mathbb{X}_{\frac{n}{c}-1}$ be set of $\mathbb{F}_2^c$, where n denotes the block size and c the size of the S-box. The vectorial division sequence of $\mathbb{X}_0, \mathbb{X}_1, \dots, \mathbb{X}_{\frac{n}{c}-1}$, denoted by $\mathbb{VDS}$, is defined as

$$\mathbb{VDS} = [\bigoplus_{x \in \mathbb{X}_0} x^u] \parallel [\bigoplus_{x \in \mathbb{X}_1} x^u] \parallel \dots \parallel [\bigoplus_{x \in \mathbb{X}_{\frac{n}{c}-1}} x^u], u \in \mathbb{F}_2^c.$$

We employ the *division sequence* ($\mathbb{DS}$) in Definition 6 to characterize the integral property of a single ciphertext cell, while the *vectorial division sequence* ($\mathbb{VDS}$) in Definition 7 is used to represent the collective integral properties over the entire ciphertext state.

4.2 Leveraging Neural Networks for Feature Detection

Constructing feature detectors using neural networks typically follows three steps: generating a dataset, designing the network architecture, and setting appropriate hyperparameters.

Generating the Dataset. Let N and M denote the sizes of the training and test sets, respectively. The dataset is generated through the following steps:

1. Label Assignment. Generate N (resp. M) binary labels Y , with approximately half labeled as 0 (negative) and the other half as 1 (positive).
2. Plaintext Set Construction. Use the Linux random number generator to sample N (or M) uniformly random plaintexts P_i . For each P_i , generate a plaintext set by varying certain bits and keeping the rest fixed.
3. Ciphertext Generation. For instances labeled $Y = 0$, replace the plaintext set with random values to eliminate structure. Then, generate N (resp. M) uniformly random keys K_i using the Linux random number generator. Encrypt each plaintext set under its corresponding key K_i to obtain the ciphertext sets.
4. Instance Formatting. Format each input instance derived from the ciphertext set according to either Definition 6 or Definition 7, depending on the required data format.

Designing Network Architecture. We adopt a simple, compact fully connected neural network with three hidden layers containing 512, 64, and 1 neurons, respectively. Each hidden layer is followed by batch normalization and a ReLU activation function. The final layer uses a sigmoid activation function for

binary classification. Although we experimented with more complex architectures, results show that this basic fully connected network is sufficient for the requirements of our study.

Settings Hyperparameters. The neural network was trained for 20 epochs using $N = 10^6$ training samples and $M = 10^5$ test samples, with a batch size of 100. Optimization was performed using the Adam optimizer, with mean squared error (MSE) as the loss function. A cyclic learning rate schedule was applied: $l_i = \alpha + \frac{(9-i) \bmod 10}{10} \cdot (\beta - \alpha)$, where $\alpha = 10^{-4}$ and $\beta = 2 \times 10^{-3}$. After each epoch, the model was saved, and the version with the lowest validation loss was selected for final evaluation on the test set. More detailed hyperparameter settings and the neural network’s training procedure can be found in [10,24].

Performance Evaluation of the Trained Neural Network. The neural network is lightweight, requiring less than one minute per training epoch. We trained several neural networks, referred to as integral neural distinguishers ($\mathcal{IND}$), on different datasets.³ The classification accuracies of these distinguishers are summarized in Tables 3, 4, 5, and 6 (The boldfaced accuracies in the table correspond to the neural distinguishers whose working mechanisms will be analyzed in detail in the following sections). Since the task involves binary classification (distinguishing ciphertexts from random data), any accuracy (Acc) above 0.5 indicates an effective distinguisher. In these tables, the $\mathcal{A}$ -set refers to the activated plaintext cells, while the $\mathcal{B}$ -set refers to the observed ciphertext cells. Configurations that did not yield a distinguisher with accuracy above 0.5 are omitted.

Table 3. Accuracy of $\mathcal{IND}$ on 7-round SKINNY-64-64 with $\mathbb{VDS}$ (1 activated cell)

$\mathcal{A}$ -set	12	13	14	15
Accuracy	0.998	0.998	0.998	0.998

Table 4. Accuracy of $\mathcal{IND}$ on 8-round SKINNY-64-64 with $\mathbb{VDS}$ (2 activated cells)

$\mathcal{A}$ -set	{4,5}	{5,6}	{6,7}	{11,12}	{12,13}	{13,14}	{14,15}
Accuracy	0.537	0.535	0.535	0.575	0.753	0.752	0.751

³ All experiments were conducted in Python 3.7.15 and TensorFlow 2.5.0 on Ubuntu 20.04. The hardware platform consisted of Intel(R) Xeon(R) Gold 6226R CPUs (2.90 GHz), 256 GB RAM, and NVIDIA RTX2080Ti GPUs (12 GB).

Table 5. Accuracy of $\mathcal{IND}$ on 8-round SKINNY-64-64 with $\mathbb{VDS}$ (1 activated cell)

$\mathcal{A}$ -set	12	13	14	15
Accuracy	0.584	0.585	0.581	0.578

Table 6. Accuracy of $\mathcal{IND}$ on 7-round SKINNY-64-64 with $\mathbb{DS}$ (1 activated cell)

$\mathcal{A}$ -set \ $\mathcal{B}$ -set	4	5	6	7	8	9	10	11
4	-	-	-	0.555	-	-	-	-
5	0.554	-	-	-	-	-	-	-
6	-	0.555	-	-	-	-	-	-
7	-	-	0.556	-	-	-	-	-
12	0.764	-	-	0.764	0.572	0.515	-	-
13	0.764	0.764	-	-	-	0.572	0.516	-
14	-	0.763	0.763	-	-	-	0.572	0.516
15	-	-	0.764	0.764	0.515	-	-	0.572

To the best of our knowledge, for SKINNY-64-64, the longest known key-independent integral distinguishers reach up to 6 rounds when one plaintext cell is activated [2], and up to 7 rounds when two plaintext cells are activated [25]. However, as shown in Tables 4 and 5, neural networks can produce 8-round integral-neural distinguishers. This observation naturally raises a fundamental question: *What features have neural networks actually learned, and why do existing automated methods fail to capture them?*

4.3 Using Boolean Functions to Explain Learned Features

Interpreting neural networks is a well-known challenge, largely due to their black-box nature. Although they perform remarkably well across a wide range of tasks, their internal decision-making processes are encoded in complex, high-dimensional, and non-linear representations that are inherently difficult to analyze. This opacity makes it challenging to understand what features the network has learned or how it arrives at its predictions. In sensitive domains such as cryptanalysis, where interpretability is just as important as accuracy, this limitation becomes particularly critical. Benamira et al. extract the parameters of one-dimensional convolutional layers from neural networks and convert them into the disjunctive normal form (DNF) of Boolean functions in order to analyze the working mechanism of differential-neural distinguishers [3].

This subsection is devoted to understanding the internal mechanisms of the integral-neural distinguisher and investigating the reasons behind its superior performance compared to existing automated search techniques. Given the large number of trained integral-neural distinguishers, we analyze only a selected subset, as shown in Table 7. We also compute their true positive rates (TPR) and true negative rates (TNR) to help understand the behavior and decision boundaries of each integral-neural distinguisher.

Table 7. The integral-neural distinguisher on SKINNY-64-64

Distinguisher	Round	Data Format	Data	$\mathcal{A}$ -set	Acc	TPR	TNR	Ref.
$\mathcal{IND}_1$	7	VDS	2^4	15th	99.8%	100%	99.6%	Table 3
$\mathcal{IND}_2$	8	VDS	2^8	{14,15}th	75.1%	99.9%	50.2%	Table 4

Since there is currently no effective approach to interpret the inner workings of the neural distinguisher, we conduct an initial, intuitive analysis based on its performance measures.

- For $\mathcal{IND}_1$, the TPR reaches 100%, indicating that the neural network has captured a simple feature such that every positive instance satisfies it. For negative instances, the probability of misclassification is $1 - 0.096 = 2^{-8}$. Since they comprise half of the dataset, the overall misclassification rate is approximately 2^{-9} , yielding an error rate of 0.2% and an accuracy of 99.8%. This suggests that the learned feature depends on only 8 bits of the 256 bits.
- For $\mathcal{IND}_2$, the TPR is close to 100%. For negative instances, the false positive rate is approximately $1 - 0.502 \approx 2^{-1}$, resulting in an overall accuracy of about 75%. This suggests the learned feature likely depends on only a single bit.

By directly examining the performance measures (Acc, TPR, TNR), we observe that the internal decision-making of the integral-neural distinguisher may depend on only a few input bits. In differential-neural cryptanalysis, Chen et al. introduced the bit sensitivity test to identify input bits that significantly affect a neural distinguisher’s performance [6]. Since the data is binary, the test works by randomly flipping one bit per input instance and observing the change in prediction accuracy. A significant drop indicates that the model relies on that bit for its decision. For details, see Algorithm 1.

We apply the bit sensitivity test to identify which bits have a significant influence on the prediction accuracy of integral-neural distinguisher $\mathcal{IND}_1$ and $\mathcal{IND}_2$.

- For $\mathcal{IND}_1$, the bit sensitivity test suggests that the integral-neural distinguisher’s decision relies primarily on 16 bits, as shown in Fig. 3. In contrast,

Algorithm 1 Bit Sensitivity Test [6]

Input: Test dataset $X = \{X_0, \dots, X_{N-1}\}$ (half positive, half negative instances); n : size of an instance X_i ; $\mathcal{ND}$: neural distinguisher

Output: Array Acc_{dec} of accuracy decreases for each bit

```
1:  $\text{Acc} \leftarrow \mathcal{ND}(X)$ 
2:  $\text{Acc}_{dec} \leftarrow \{\}$ 
3: for  $i = 0$  to  $n - 1$  do
4:    $X_{masked} \leftarrow \{\}$ 
5:   for  $j = 0$  to  $N - 1$  do
6:     Generate a random mask  $\eta \in \{0, 1\}$ 
7:      $X_{masked}^j \leftarrow X^j \oplus (\eta \ll i)$  ▷ Randomize bit  $i$ 
8:   end for
9:   Append  $\text{Acc} - \mathcal{ND}(X_{masked})$  to  $\text{Acc}_{dec}$ 
10: end for
11: return  $\text{Acc}_{dec}$ 
```

analysis of the performance matrix indicates that the distinguisher depends on only 8 bits. This highlights a discrepancy between the two methods.

- For $\mathcal{IND}_2$, the bit sensitivity test suggests that the integral-neural distinguisher primarily relies on 3 bits. However, analysis of the performance matrix indicates that its decision depends on only a single bit. This again highlights a discrepancy between the two methods.

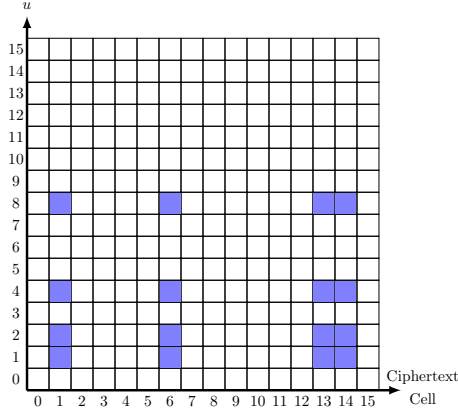


Fig. 3. Bit sensitivity test of the $\mathcal{IND}_1$

The inconsistency stems from the fact that the bit-sensitivity test merely identifies input bits that influence the model's decisions, without uncovering the specific decision rules or the logic derived from those bits. To address this limitation, we employ Boolean functions as an effective tool for analyzing and ex-

plaining the working mechanism. Boolean functions can serve as effective feature interpreters for integral-neural distinguishers for the following key reasons:

1. **Binary Inputs and Outputs:** Both the inputs (e.g., $\mathbb{DS}$ or $\mathbb{VDS}$) and outputs of $\mathcal{IND}$ are binary. Boolean functions are well-suited to modeling binary-to-binary mappings, making them a natural choice for characterizing the input-output behavior of neural networks in cryptographic tasks.
2. **Feasible Input Size for Boolean Modeling:** Although the full $\mathbb{VDS}$ can contain up to 256 bits, which makes exhaustive Boolean analysis infeasible, the bit sensitivity test shows that $\mathcal{IND}$ typically depends on only a small number of critical bits. This reduced input dimensionality allows us to construct the full truth table and derive a precise Boolean expression.
3. **Explicit Logical Rule Extraction:** Despite the high-dimensional and non-linear nature of neural networks, Boolean functions—expressed in forms such as CNF (Conjunctive Normal Form) or ANF (Algebraic Normal Form)—enable the extraction of human-interpretable logical rules. These rules reveal how the neural network combines input bits to make decisions.
4. **Model Compression and Verifiability:** Once extracted, a Boolean function can serve not only as an explanation of the neural network’s behavior but also as a lightweight and deterministic alternative to the model. This is particularly valuable in cryptographic applications, where interpretability, transparency, and verifiability are crucial.

Transforming an Integral-neural Distinguisher to a Boolean Function.

We first retrain the integral-neural distinguishers on new instances constructed from sensitive bits identified through bit-sensitivity testing. The retrained model is then treated as an oracle, from which we obtain the truth table of the corresponding Boolean function by exhaustively enumerating all possible inputs. To make the learned features more interpretable, we use **Espresso**⁴ to derive the CNF representation of the Boolean function. The complete procedure is described in Algorithm 2.

We apply Algorithm 2 to extract the CNF representation of the retrained model corresponding to $\mathcal{IND}_1$, shown as follows:

$$\begin{aligned}
& (\neg b_1^1 \vee b_{13}^1) \wedge (b_1^1 \vee \neg b_{13}^1) \wedge (\neg b_1^2 \vee b_{13}^2) \wedge (b_1^2 \vee \neg b_{13}^2) \\
& \wedge (\neg b_1^4 \vee b_{13}^4) \wedge (b_1^4 \vee \neg b_{13}^4) \wedge (\neg b_1^8 \vee b_{13}^8) \wedge (b_1^8 \vee \neg b_{13}^8) \\
& \wedge (\neg b_6^1 \vee b_{14}^1) \wedge (b_6^1 \vee \neg b_{14}^1) \wedge (\neg b_6^2 \vee b_{14}^2) \wedge (b_6^2 \vee \neg b_{14}^2) \\
& \wedge (\neg b_6^4 \vee b_{14}^4) \wedge (b_6^4 \vee \neg b_{14}^4) \wedge (\neg b_6^8 \vee b_{14}^8) \wedge (b_6^8 \vee \neg b_{14}^8).
\end{aligned}$$

Here, b_i^u denotes the bit at position u within the i th ciphertext cell of the $\mathbb{VDS}$. For the CNF to evaluate to true, all clauses must be simultaneously satisfied. This condition implies that the following XOR relations are always equal to zero:

$$b_1^1 \oplus b_{13}^1, b_1^2 \oplus b_{13}^2, b_1^4 \oplus b_{13}^4, b_1^8 \oplus b_{13}^8, b_6^1 \oplus b_{14}^1, b_6^2 \oplus b_{14}^2, b_6^4 \oplus b_{14}^4, b_6^8 \oplus b_{14}^8.$$

⁴ <https://github.com/classabbyamp/espresso-logic>

Algorithm 2 CNF Representation of the Integral-neural Distinguisher

Input: Retrained integral-neural distinguisher $\mathcal{IND}$; input size d

Output: CNF representation $\text{CNF}_{\mathcal{IND}}$

```
1: TT  $\leftarrow []$  ▷ Initialize truth table
2: for each  $x \in \{0, 1\}^d$  do
3:   if  $\mathcal{IND}(x) > 0.5$  then
4:     Append 1 to TT
5:   else
6:     Append 0 to TT
7:   end if
8: end for
9:  $\mathcal{BF} \leftarrow \text{BOOLEANFUNCTION}(\text{TT})$  ▷ Construct Boolean function
10:  $\text{CNF}_{\mathcal{IND}} \leftarrow \text{Espresso}(\mathcal{BF})$  ▷ Minimize and output CNF
11: return  $\text{CNF}_{\mathcal{IND}}$ 
```

Therefore, we can conclude that $\mathcal{IND}_1$ has learned an integral property based on a linear combination of ciphertext bits, corresponding to Definition 4 where the function g is a linear function.

Based on this observation, we derive a 7-round integral distinguisher for SKINNY, and empirical statistical tests support its effectiveness. Moreover, this property holds not only for SKINNY-64-64 but also for SKINNY-128-128. The specific form of the distinguisher is presented as follows:

Distinguisher 1 (Key-independent Integral Distinguisher Based on Linear Combination of Ciphertext Bits on 7-round SKINNY- n - n). *For 7-round SKINNY- n - n , when the 15th plaintext cell is activated, the following relation always deterministically holds:*

$$\bigoplus_{x_1 \in \mathbb{X}_1} \pi_u(x_1) \oplus \bigoplus_{x_2 \in \mathbb{X}_2} \pi_u(x_2) = 0,$$

for all u with $\text{wt}(u) = 1$ and ciphertext cell pairs $(\mathbb{X}_1, \mathbb{X}_2) \in \{(1, 13), (6, 14)\}$.

By applying Algorithm 2 to $\mathcal{IND}_2$, we derive an 8-round integral distinguisher for SKINNY-64-64 that holds when two plaintext bytes are activated, and empirical statistical tests support its effectiveness.

Distinguisher 2 (Key-independent Integral Distinguishers Based on Linear Combination of Ciphertext Bits on 8-round SKINNY-64-64). *If 14th and 15th plaintext cells are activated, then the linear combination of ciphertext bits $b_{28} \oplus b_{44} \oplus b_{60}$ is deterministically balanced.*

4.4 Improved Key Recovery Attack on SKINNY

At CRYPTO 2016, Beierle et al. identified a 6-round integral distinguisher from single-cell activation, enabling 14-round key-recovery attacks on SKINNY- n - n [2]. Zhang et al. [25] proposed a 16-round integral attack on SKINNY-128-128, but

their analysis focused only on the key guessing space and did not provide a full evaluation of the time complexity. Under the chosen-tweak setting, Hadipour et al. [12] mounted an 18-round key-recovery attack on SKINNY- n - n by tweaking a single key byte, leveraging an integral distinguisher derived from a zero-correlation property.

We use a 7-round integral distinguisher (**Distinguisher 1**) based on a linear combination of ciphertext bits to perform a key-recovery attack. Although **Distinguisher 2** covers one more round than **Distinguisher 1**, the key-recovery attack based on it has a higher complexity than the attack using **Distinguisher 1**, since it involves more plaintext and ciphertext cells. The detailed procedure is described as follows.

The Key Recovery Attack for SKINNY- n - n . The overall procedure of the key-recovery attack is illustrated in Fig 4. The target cipher E is divided into three parts: E_0 covers rounds $(0 \rightarrow \dots \rightarrow 3)$, E_1 (**Distinguisher 1** in Section 4.3) covers rounds $(4 \rightarrow \dots \rightarrow 10)$, and E_2 covers rounds $(11 \rightarrow \dots \rightarrow 15)$. In the following, we denote the input state of the S-box and MixColumns in the r -th round by x_r and z_r , respectively, and use $[j]$ to represent the j -th cell of a state. TK denotes the tweakkey state.

We first choose $2^{12 \times c}$ plaintexts by taking a fixed starting point and varying 12 bytes in the input of the second round. We can trivially prepend one round because adding the equivalent tweakkey does not alter the input structure. Instead of guessing the keys for E_0 , we simply use all $2^{12 \times c}$ plaintexts, as illustrated in [9]. These encryptions consist of $2^{11 \times c}$ groups of 2^c encryptions that vary only in a single byte $\mathbb{Z}$ of the 4th round, sum this value over all the $2^{12 \times c}$ encryptions, and check for a zero result. For the key recovery, we separately recover the sums in $x_{11}[1]$ (■, branch 1) and $x_{11}[13]$ (■, branch 2) using the partial-sum technique [9] and merge the results following the meet-in-the-middle approach [18]. States that affect both branches are indicated in ■. The procedures for the two sums are summarized in Table 8 and Table 9.

Table 8. Key recovery steps and complexities for branch 1

Step	Guessed	$K \times D = \text{Mem}$	Time	Stored Texts
0	–	$2^0 \times 2^{11 \times c}$	$2^{12 \times c - 5.4}$	$z_{15}[0, 2, 3, 5, 6]; x_{15}[9, 12, 13, 14, 15]; x_{15}[8] \oplus x_{15}[12]$
1	TK[8]	$2^c \times 2^{10 \times c}$	$2^{12 \times c - 7}$	$x_{14}[13]; z_{15}[2, 3, 5, 6]; x_{15}[9, 13, 14, 15]; x_{15}[8] \oplus x_{15}[12]$
2	TK[12]	$2^{2 \times c} \times 2^{9 \times c}$	$2^{12 \times c - 8}$	$x_{14}[13]; z_{14}[4]; z_{15}[2, 3, 6]; x_{15}[9, 13, 14, 15]$
3	TK[10]	$2^{3 \times c} \times 2^{8 \times c}$	$2^{12 \times c - 7}$	$x_{14}[13, 15]; z_{14}[4]; z_{15}[3, 6]; x_{15}[9, 14, 15]$
4	TK[11]	$2^{4 \times c} \times 2^{7 \times c}$	$2^{12 \times c - 7}$	$x_{14}[12, 13, 15]; z_{14}[4]; z_{15}[6]; x_{15}[9, 15]$
5	TK[13]	$2^{5 \times c} \times 2^{6 \times c}$	$2^{12 \times c - 7}$	$x_{14}[12, 13]; z_{14}[1, 4, 5]; x_{14}[11] \oplus x_{14}[15]$
6	TK[6]	$2^{6 \times c} \times 2^{5 \times c}$	$2^{12 \times c - 7}$	$x_{13}[10]; x_{14}[13]; z_{14}[1, 4]; x_{14}[11] \oplus x_{14}[15]$
7	TK[0]	$2^{7 \times c} \times 2^{3 \times c}$	$2^{12 \times c - 7}$	$x_{13}[10] \oplus x_{13}[14]; z_{14}[4]; x_{14}[11] \oplus x_{14}[15]$
8	TK[1]	$2^{8 \times c} \times 2^{1 \times c}$	$2^{11 \times c - 7}$	$z_{12}[6]$
9	TK[7]	$2^{9 \times c} \times 2^c$	$2^{10 \times c - 7}$	$x_{11}[1]$

Table 9. Key recovery steps and complexities for branch 2

Step	Guessed	$K \times D = \text{Mem}$	Time	Stored Texts
0	–	$2^0 \times 2^{10 \times c}$	$2^{12 \times c - 5.4}$	$z_{15}[0, 1, 4, 5, 7]; x_{15}[12, 13, 15]; x_{15}[8] \oplus x_{15}[12]; x_{15}[10] \oplus x_{15}[14]$
1	$TK[8]$	$2^c \times 2^{9 \times c}$	$2^{11 \times c - 7}$	$x_{14}[13]; z_{15}[1, 4, 5, 7]; x_{15}[13, 15]; x_{15}[8] \oplus x_{15}[12]; x_{15}[10] \oplus x_{15}[14]$
2	$TK[12]$	$2^{2 \times c} \times 2^{8 \times c}$	$2^{11 \times c - 8}$	$x_{14}[13]; z_{14}[4]; z_{15}[1, 4, 7]; x_{15}[13, 15]; x_{15}[10] \oplus x_{15}[14]$
3	$TK[9]$	$2^{3 \times c} \times 2^{7 \times c}$	$2^{11 \times c - 7}$	$x_{14}[13, 14]; z_{14}[4]; z_{15}[4, 7]; x_{15}[15]; x_{15}[10] \oplus x_{15}[14]$
4	$TK[14]$	$2^{4 \times c} \times 2^{7 \times c}$	$2^{11 \times c - 8}$	$x_{14}[13, 14]; z_{14}[2, 4, 6]; z_{15}[4]; x_{15}[15]$
5	$TK[15]$	$2^{5 \times c} \times 2^{5 \times c}$	$2^{12 \times c - 7}$	$x_{14}[14]; z_{14}[2, 4, 6]; x_{14}[9] \oplus x_{14}[13]$
6	$TK[3]$	$2^{6 \times c} \times 2^{4 \times c}$	$2^{11 \times c - 8}$	$z_{13}[5]; x_{14}[14]; z_{14}[2, 4]$
7	$TK[4]$	$2^{7 \times c} \times 2^{3 \times c}$	$2^{11 \times c - 7}$	$x_{13}[15]; z_{13}[5]; z_{14}[4]$
8	$TK[1]$	$2^{8 \times c} \times 2^c$	$2^{11 \times c - 5.4}$	$x_{11}[13]$

Complexity Analysis. We use the number of S-box lookups as a unit for the time complexity, as in previous attacks.

Therefore, a single zero-sum test yields 2^{60} key candidates with a time complexity of $2^{44.82}$ for SKINNY-64-64, which is less than the query complexity of 2^{48} . By repeating the attack 4 times, the key candidate space can be reduced to 2^{48} . Thus, the overall time complexity of the attack for SKINNY-64-64 is 2^{50} , with a data complexity of 2^{50} .

Using a similar procedure, the overall time complexity of the attack for SKINNY-128-128 is 2^{98} , with a data complexity of 2^{98} .

The Key Recovery Attack for SKINNY- n - $2n$ and SKINNY- n - $3n$. Since the 7-round integral distinguisher is independent of the key, it can be applied to SKINNY- n with different key lengths.

- Using a similar procedure, we can perform an 18-round key recovery attack on SKINNY- n - $2n$. The attack process is shown in Appendix A Fig. 5. The data complexity is 2^{12c} . We guess the round keys of the last two rounds to obtain the state of z_{15} with a time complexity of 2^{28c} , and then perform key recovery using techniques similar to those in the attack on SKINNY- n - n . By repeating this process 4 times, the overall attack complexity amounts to $4 \times (2^{112} + 2^{44.82}) \approx 2^{114}$ for SKINNY-64-128, yielding 2^{112} key candidates; while for SKINNY-128-256 it becomes $4 \times (2^{224} + 2^{92.76}) \approx 2^{226}$, yielding 2^{224} key candidates.
- Using a similar procedure, we can perform a 20-round key recovery attack on SKINNY- n - $3n$. We guess the round keys of the last two rounds. The attack process is shown in Appendix A Fig. 6. By repeating this process 4 times, the overall attack complexity amounts to $4 \times (2^{44 \times 4} + 2^{44.82}) \approx 2^{178}$ for SKINNY-64-192, yielding 2^{176} key candidates; while for SKINNY-128-384 it becomes $4 \times (2^{44 \times 8} + 2^{92.76}) \approx 2^{354}$, yielding 2^{352} key candidates.

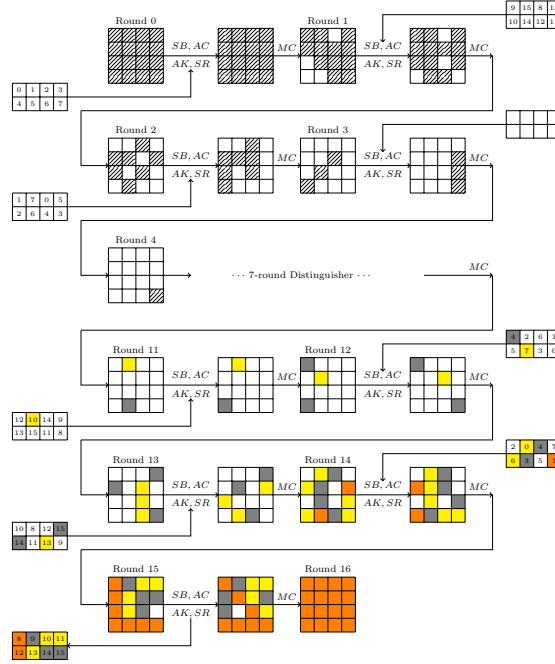


Fig. 4. 16-round key recovery attack for SKINNY- n - n in single tweak setting

5 Neural Network-Navigated Optimization of the Automated Search Model

In this section, the neural network-derived distinguisher enables us to avoid enumerating a large number of candidate linear combinations of ciphertext bits. This guidance optimizes the automated search model and enables the discovery of an 11-round integral distinguisher for SKINNY- n - n , and achieves the upper bound for integral distinguishers of non-standard SKINNY.

5.1 Discussion of Integral Distinguishers from Neural and Automated Approaches

After converting the integral-neural distinguishers into Boolean functions, we find that the neural networks have effectively learned integral properties based on the linear combination of ciphertext bits. However, the integral distinguishers derived from the neural networks had not been discovered by existing automated search methods. A comparison of existing automated search model-derived and neural network (NN)-derived integral distinguishers for SKINNY-54-64 is presented in Table 10.

The inability of existing automated search models to reproduce the results achieved by neural networks may be attributed to the following two factors:

Table 10. Comparison of model-derived and NN-derived Integral Distinguishers

Round	Data Complexity	Method	Feature Type	Reference
7	2^8	Model-derived	Single Bit	[25]
	2^4	NN-derived	Linear Combination of Bits	Distinguisher 1
8	2^{15}	Model-derived	Linear Combination of Bits	[7]
	2^8	NN-derived	Linear Combination of Bits	Distinguisher 2

1. The number of possible linear combinations of ciphertext bits is too large, making it infeasible for automated search models to exhaustively explore all possibilities. For *SKINNY-64-64*, the total number of possible linear combinations of ciphertext bits is as high as 2^{64} . To make the search tractable, Derbez and Fouque et al. only considered combinations within the same column, reducing the space to $16 \times (2^4 - 1)$ possibilities [7].
2. Existing automated models fall short in capturing all integral-related features, limiting their ability to discover certain distinguishers that neural networks can detect. Derbez and Fouque et al. improved the precision of the division property by computing propagation tables for 16-bit Super-Sboxes and applying linear combinations of ciphertext bits to search for integral distinguishers in *SKINNY* [7]. In a follow-up work [8], they employed the division property with two subsets to extend the analysis. However, neither study adopted more advanced techniques such as the three-subset division property [21] or monomial prediction [15].

While increasing model precision may improve results, it also significantly increases computational complexity. This is especially true when analyzing integral properties involving linear combinations of ciphertext bits. High-precision models can quickly become computationally infeasible, making it difficult to produce results within a reasonable time frame. Neural networks, however, can directly identify specific expressions of integral properties (e.g., particular linear combinations of ciphertext bits), which allows us to bypass the challenge of exhaustive search and focus instead on enhancing the precision of the search model.

5.2 Strategy for Balancing Precision and Complexity

At ASIACRYPT 2021, Hu et al. addressed the challenge of exact algebraic normal form (ANF) recovery for an r -round cipher, where $r = p + q$, by decomposing the task into two phases: a p -round forward monomial prediction and a q -round backward monomial expansion [14]. In this framework, each ciphertext bit is represented using variables from an intermediate state. By fixing the forward-phase solving time and tuning q , they ensured computational efficiency. However, this

model is not well-suited for searching integral distinguishers based on linear combinations of ciphertext bits, as the enormous search space leads to prohibitively high computational complexity.

To address this limitation, the p -round forward monomial prediction is replaced with a more efficient, though less precise, division property with two subsets. The model's overall precision is then enhanced by increasing q in the backward expansion phase; however, the search complexity grows rapidly as q increases. In **SKINNY-64-64**, the **MixColumns** and **SubCells** layers can introduce up to 3 and 9 monomials per input monomial, respectively. Consequently, each additional round of backward expansion increases the number of monomials to be examined by a factor of 3×9 . The framework evaluates the integral property of each monomial generated after q rounds of backward expansion. The core procedure of the refined automated search model is summarized as follows:

Let $\mathbf{x} = (x_0, \dots, x_{n-1})$ be the plaintext, $\mathbf{s} = (s_0, \dots, s_{n-1})$ the intermediate state after p rounds, $\mathbf{b} = (b_0, \dots, b_{n-1})$ the ciphertext after $p + q$ rounds, and $\mathbf{k} = (k_0, \dots, k_{n-1})$ the secret key in **SKINNY- n - n** . Bits are indexed from left to right. The encryption from round r_i to r_j is denoted as $E_{\mathbf{k}}^{(r_i, r_j)}$, so we have: $\mathbf{b} = E_{\mathbf{k}}^{(0, p+q)}(\mathbf{x}) = E_{\mathbf{k}}^{(p, p+q)}(\mathbf{s})$. The initial division property is denoted by $\mathbf{d}_0$, and the set of division properties after p rounds is represented by $\mathbb{D}_p$.

1. q -round backward monomial expansion. Given a combination of ciphertext bits $\mathcal{C}(\mathbf{b})$, recursively expand it backward into its ANF over the p -round intermediate variables $\mathbf{s}$:

$$\mathcal{C}(\mathbf{b}) = \mathcal{C}(E_{\mathbf{k}}^{(0, p+q)}(\mathbf{x})) = \mathcal{C}(E_{\mathbf{k}}^{(p, p+q)}(\mathbf{s})) = \sum_i \pi_{\mathbf{u}_i}(\mathbf{s}) \pi_{\mathbf{v}_i}(\mathbf{k}).$$

Only the balanced properties of $\pi_{\mathbf{u}_i}(\mathbf{s})$ are considered, treating $\pi_{\mathbf{v}_i}(\mathbf{k})$ as constants. To accelerate the process, **Reducing Rule 1** from Lambin et al. [17] is applied to reduce the number of monomials under examination.

2. p -round forward modeling using the division property with two subsets. The bit-based division property is propagated forward over p rounds, and the resulting properties are stored in the set $\mathbb{D}_p$. The MILP-based modeling follows the framework of [23]. We omit the details and solve the resulting MILP instances using the Gurobi optimizer [11]. We note that the **MixColumns** operation in **SKINNY** consists solely of XOR operations, so the bits within each column can be processed independently. Exploiting this property, **MixColumns** can be decomposed into 16 identical functions, each mapping 4 input bits to 4 output bits, where the input and output correspond to the four bits in the same column of the state matrix. Consequently, **MixColumns** can be treated as a 4×4 S-box and modeled using standard S-box techniques, rather than being encoded directly from its gate-level circuit. This modeling choice simplifies the formulation and improves the efficiency of automated solvers.

3. Checking the balance property of the combination of ciphertext bits. For each monomial $\pi_{\mathbf{u}_i}(\mathbf{s})$ in the ANF, the balance condition is determined as:

$$\sum_i \pi_{\mathbf{u}_i}(\mathbf{s}) = \begin{cases} \text{unknown} & \text{if there exists } \mathbf{d} \in \mathbb{D}_p \text{ such that } \mathbf{u}_i \succeq \mathbf{d}, \\ 0 & \text{otherwise.} \end{cases}$$

If all $\pi_{\mathbf{u}_i}(\mathbf{s})$ are balanced, then the combination of ciphertext bits $\mathcal{C}(\mathbf{b})$ is considered balanced.

Reducing Rule 1 [17]. Given a monomial set $\mathbb{U}$, if $\mathbf{u}_i \succeq \mathbf{u}_j$ for some $\mathbf{u}_i, \mathbf{u}_j \in \mathbb{U}$, then $\mathbf{u}_i$ also dominates all monomials dominated by $\mathbf{u}_j$. Therefore, it is sufficient to check only the reduced set: $\mathbb{U}' = \{\mathbf{u} \in \mathbb{U} \mid \nexists \mathbf{v} \in \mathbb{U} \text{ such that } \mathbf{v} \succeq \mathbf{u}\}$.

The algorithm for checking the balanced property of a combination of ciphertext bits is presented in Algorithm 3.

Algorithm 3 Search for Key-independent Integral Distinguisher Based on Combination of Ciphertext Bits

Input: A combination $\mathcal{C}(\mathbf{b})$, round parameters p and q , input division property $\mathbf{d}_0$

Output: True if $\mathcal{C}(\mathbf{b})$ is balanced; False otherwise

```

1:  $\mathbb{U} \leftarrow \text{BackwardExpand}(\mathcal{C}(\mathbf{b}), q)$ 
2:  $\mathbb{U}' \leftarrow \text{Apply ReducingRule 1}(\mathbb{U})$ 
3: for each monomial  $\mathbf{u}_i \in \mathbb{U}'$  do
4:    $\mathcal{M} \leftarrow \text{Model}(p, \mathbf{d}_0, \mathbf{u}_i)$ 
5:    $\mathcal{M}.\text{optimize}()$ 
6:   if  $\mathcal{M}.\text{status} = \text{Optimal}$  then
7:     return False
8:   end if
9: end for
10: return True

```

Guiding the Search Using the NN-Derived Integral Distinguisher.

Since the explicit form of the integral distinguisher has already been derived from the neural network, our objective is merely to verify whether the refined automated search model can rediscover this specific distinguisher, without the need to consider all possible linear combinations.

- For Distinguisher 1, we identify the integral property of $b_4 \oplus b_{52}$. Specifically, we perform a one-round backward monomial expansion followed by six rounds of forward modeling to rediscover this distinguisher. As shown in Equation (1), we verify whether each monomial is balanced:

$$b_4 \oplus b_{52} = s_{56}s_{57} \oplus s_{56} \oplus s_{57} \oplus s_{59} \oplus 1 \quad (1)$$

- For Distinguisher 2, we identify the integral property of $b_{28} \oplus b_{44} \oplus b_{60}$. To rediscover this distinguisher, we apply a two-round backward monomial

expansion followed by six rounds of forward modeling. As shown in Equation (2), we then examine whether each monomial in the resulting polynomial is balanced:

$$b_{28} \oplus b_{44} \oplus b_{60} = s_9 s_{10} k_{28} \oplus s_8 s_9 k_{29} \oplus s_8 k_{28} \oplus s_9 k_{28} \oplus s_{10} k_{28} \oplus s_8 k_{29} \oplus s_9 k_{29} \oplus s_{11} k_{29} \oplus k_{28} k_{29} \oplus s_{10} \oplus k_{31} \oplus k_{36} \quad (2)$$

In our refined automated search framework, setting $q = 2$ suffices to reproduce the NN-derived integral distinguishers, indicating that the model achieves the required precision. We therefore shift our focus to employing the refined search model independently to explore longer-round integral distinguishers for SKINNY.

Exploration of Longer-Round Integral Distinguishers. For SKINNY- n - n , there are 2^n possible linear combinations of ciphertext bits, rendering exhaustive search infeasible. To control the search complexity, we follow the approach of Derbez and Fouque et al. [7] and restrict the analysis to linear combinations confined to a single column. Specifically, this results in $4 \times c \times (2^c - 1)$ possible combinations, where c denotes the number of S-boxes. As a result, by setting $q = 1$, we successfully identify an integral distinguisher for 11-round SKINNY-64-64 (resp. SKINNY-128-128) by activating only 60 (resp. 120) plaintext bits. To the best of our knowledge, this represents the first integral distinguisher for 11 rounds of SKINNY- n - n . The explicit form of the integral distinguisher is given as follows:

Distinguisher 3 (Key-independent Integral Distinguisher Based on Linear Combination of Ciphertext Bits on 11-round SKINNY-64-64). *If the last 60 plaintext bits are activated, then the linear combination of ciphertext bits $b_i \oplus b_{i+16} \oplus b_{i+32}$ for $i \in \{16, 17, \dots, 31\}$ and $b_{12} \oplus b_{60}$ are balanced.*

Distinguisher 4 (Key-independent Integral Distinguisher Based on Linear Combination of Ciphertext Bits on 11-round SKINNY-128-128). *If the last 120 plaintext bits are activated, then the linear combination of ciphertext bits $b_i \oplus b_{i+16} \oplus b_{i+32}$ for $i \in \{26, \dots, 63\}$ and $b_{26} \oplus b_{122}$ are balanced.*

Achieving the Upper Limit of Integral Distinguishers for Non-standard SKINNY. In ASIACRYPT 2021, Hebborn et al. [13] demonstrated that, under the assumptions of $n-1$ active plaintext bits, independent round keys, and full-state key XOR, no 13-round integral distinguisher exists for non-standard SKINNY-64 based on linear combinations of ciphertext bits. Since SKINNY does not employ whitening keys, any 12-round distinguisher can be trivially extended to 13 rounds. Therefore, excluding such trivial extensions, the longest possible key-independent integral distinguisher spans 11 rounds. Using Algorithm 3 under the same assumptions, we successfully reach this upper bound by activating only 60 plaintext bits, identifying an 11-round integral distinguisher.

6 Key-dependent (Weak-key) Integral Distinguisher

Section 4.3 showed that neural networks can capture integral properties from linear combinations of ciphertext bits, leading to the refinement of the auto-

mated search model in Section 5 and the attainment of the upper bound of integral distinguishers for SKINNY. Next, we analyze the internal mechanisms of the integral-neural distinguishers in Tables 5 and 6.

6.1 Probabilistic Integral Property Based on Linear Combinations

Table 11 shows an 8-round integral-neural distinguisher for SKINNY-64-64, referred to as $\mathcal{IND}_3$. In this case, u is constrained to have Hamming weight 1, so the corresponding VDS contains 64 elements.

Table 11. The integral-neural distinguisher $\mathcal{IND}_3$ on SKINNY-64-64

Distinguisher	Round	Data Format	Data	$\mathcal{A}$ -set	Acc	TPR	TNR	Ref.
$\mathcal{IND}_3$	8	VDS	2^4	15th	57.8%	54.6%	61.0%	Table 5

To analyze the internal behavior of the integral-neural distinguisher, a bit-sensitivity test is carried out, and the CNF representation of the retrained model for $\mathcal{IND}_3$ is obtained, as presented below:

$$\begin{aligned} & (\neg b_4^8 \vee \neg b_8^8 \vee \neg b_{12}^8) \wedge (b_4^8 \vee b_8^8 \vee \neg b_{12}^8) \wedge (b_4^8 \vee \neg b_8^8 \vee b_{12}^8) \wedge (\neg b_4^8 \vee b_8^8 \vee b_{12}^8) \\ & \wedge (\neg b_7^8 \vee \neg b_{11}^8 \vee \neg b_{15}^8) \wedge (b_7^8 \vee b_{11}^8 \vee \neg b_{15}^8) \wedge (b_7^8 \vee \neg b_{11}^8 \vee b_{15}^8) \wedge (\neg b_7^8 \vee b_{11}^8 \vee b_{15}^8) \end{aligned}$$

For the CNF to evaluate to true, all clauses must be satisfied simultaneously. This implies that the following XOR relations always evaluate to 0:

$$b_4^8 \oplus b_8^8 \oplus b_{12}^8, \quad b_7^8 \oplus b_{11}^8 \oplus b_{15}^8.$$

From the extracted CNF, it can be observed that $\mathcal{IND}_3$ captures an integral property corresponding to a linear combination of ciphertext bits. This raises a critical question: *why is the TPR not close to 100%*? If the learned property was a deterministic integral property, then all ciphertexts from the corresponding structure would satisfy the condition, yielding a TPR near 100%.

The Origin of the Probabilistic Behavior. To investigate this issue, the equation $b_4^8 \oplus b_8^8 \oplus b_{12}^8 = 0$ was evaluated over 10^7 positive instances, and it holds for about 62.7% of the samples. This suggests that the integral-neural distinguisher captures a *probabilistic* rather than a deterministic integral property. To trace the origin of this behavior, a 2-round backward monomial expansion is performed, yielding the ANF expression $b_{16} \oplus b_{32} \oplus b_{48}$, which corresponds to the equation $b_4^8 \oplus b_8^8 \oplus b_{12}^8$ rewritten using ciphertext bit indices. The resulting polynomial is given in Equation (3), where s represents the input state of the last two rounds.

$$\begin{aligned}
b_{16} \oplus b_{32} \oplus b_{48} = & \underbrace{s_{14}(1 \oplus k_{16}) \oplus s_{13}(k_{16} \oplus k_{17}) \oplus s_{12}(k_{16} \oplus k_{17}) \oplus s_{15}k_{17}}_{\text{balanced}} \\
& \oplus \underbrace{s_{16}k_{17} \oplus k_{19} \oplus k_{48}}_{\text{balanced}} \oplus \underbrace{s_{13}s_{14}k_{16} \oplus s_{12}s_{13}k_{17}}_{\text{unknown, key-dependent}}.
\end{aligned} \tag{3}$$

As shown in Equation (3), the two unknown monomials involve key-dependent terms. Assuming the key is uniformly random, the probability that both k_{16} and k_{17} are equal to 0 is 0.25. In this case, the expression $s_{13}s_{14}k_{16} \oplus s_{12}s_{13}k_{17}$ evaluates to 0, and the corresponding monomials are balanced. When k_{16} and k_{17} are not both 0 (with probability 0.75), the probability that $s_{13}s_{14}k_{16} \oplus s_{12}s_{13}k_{17}$ is balanced is 0.5. Therefore, the overall probability that $b_{16} \oplus b_{32} \oplus b_{48}$ is balanced can be estimated as: $0.25 \times 1 + 0.75 \times 0.5 = 0.625$, which closely matches the empirical result of 0.627.

Fundamentally, deep learning is a form of statistical learning. From this perspective, neural networks implicitly account for specific key values and learn key-dependent integral properties in a statistical sense, ultimately expressing them as a probabilistic integral property. Based on this observation, we give a key-dependent (weak-key) integral distinguisher, as presented in Distinguisher 5. A similar 8-round integral distinguisher also exists for SKINNY-128-128, as shown in Distinguisher 6.

Distinguisher 5 (Key-dependent Integral Distinguisher Based on Linear Mapping of Ciphertext Bits on 8-round SKINNY-64-64). *If the 15th plaintext cell is activated, then the linear mapping of ciphertext bits $b_{16} \oplus b_{32} \oplus b_{48}$ and $b_{28} \oplus b_{44} \oplus b_{60}$ are balanced, dependent on two key bits.*

Distinguisher 6 (Key-dependent Integral Distinguisher Based on Linear Mapping of Ciphertext Bits on 8-round SKINNY-128-128). *If the 15th plaintext cell is activated, then the linear mapping of ciphertext bits $b_{33} \oplus b_{65} \oplus b_{97}$ and $b_{57} \oplus b_{89} \oplus b_{121}$ are balanced, dependent on two key bits.*

Refined Automatic Search for Key-Dependent Integral Distinguisher
Building upon Algorithm 3, each unknown monomial is examined individually by extracting the involved key bits and estimating the probability that the corresponding linear combination of ciphertext bits is balanced when both key bits and constant plaintext bits are randomly assigned, as described in Algorithm 4. When key bits appear in an unknown monomial, **Reducing Rule 1** is no longer sufficient. For instance, consider two monomials $\pi_{\mathbf{u}_i}(\mathbf{s})\pi_{\mathbf{v}_i}(\mathbf{k})$ and $\pi_{\mathbf{u}_j}(\mathbf{s})\pi_{\mathbf{v}_j}(\mathbf{k})$ with $\mathbf{u}_i \succeq \mathbf{u}_j$, $\mathbf{v}_i \neq 0$, $\mathbf{v}_j = 0$. In this case, $\pi_{\mathbf{u}_j}(\mathbf{s})\pi_{\mathbf{v}_j}(\mathbf{k})$ cannot be discarded, since it remains unresolved. Conversely, if $\mathbf{u}_i = \mathbf{u}_j$ and $\mathbf{v}_i \preceq \mathbf{v}_j$ for all relevant monomials, the pair $(\mathbf{u}_j, \mathbf{v}_j)$ may be safely disregarded. This refinement is formalized as **Reducing Rule 2**. If Algorithm 4 returns a probability of 1, the corresponding integral distinguisher is key-independent. Hence, the algorithm can be used to search for both key-independent and key-dependent integral distinguishers within a unified framework.

Algorithm 4 Search for Key-Dependent Integral Distinguisher Based on Combination of Ciphertext Bits

Input: A combination $\mathcal{C}(\mathbf{b})$, round parameters p and q , input division property $\mathbf{d}_0$
Output: Estimated probability that $\mathcal{C}(\mathbf{b})$ is balanced

```

1:  $\mathbb{U} \leftarrow \text{BackwardExtension}(\mathcal{C}(\mathbf{b}), q)$ 
2:  $\mathbb{U}' \leftarrow \text{ApplyReducingRule } 2(\mathbb{U})$ 
3:  $\mathbb{K} \leftarrow \emptyset$  ▷ Set of key bit indices in the unknown monomials
4: for each  $(\mathbf{u}_i, \mathbf{v}_i) \in \mathbb{U}'$  do
5:   if  $\mathbf{u}_i = \mathbf{0}$  then
6:     continue
7:   end if
8:    $\mathcal{M} \leftarrow \text{Model}(p, \mathbf{d}_0, \mathbf{u}_i)$ 
9:    $\mathcal{M}.\text{optimize}()$ 
10:  if  $\mathcal{M}.\text{status}$  is Optimal then
11:    if  $\mathbf{v}_i = \mathbf{0}$  then
12:      return 0
13:    end if
14:     $\mathbb{K}.\text{add}(\{j \mid \mathbf{v}_i[j] = 1\})$  ▷ Collect key bits involved
15:  end if
16: end for
17: return  $2^{-|\mathbb{K}|} + (1 - 2^{-|\mathbb{K}|}) \times 0.5$  ▷ Key bits = 0 with prob  $2^{-|\mathbb{V}|}$ 

```

Reducing Rule 2. Given a set of monomials $\mathbb{U} = \{(\mathbf{u}_i, \mathbf{v}_i)\}$, the reduced set is defined as: $\mathbb{U}' = \{(\mathbf{u}_i, \mathbf{v}_i) \in \mathbb{U} \mid \nexists (\mathbf{u}_j, \mathbf{v}_j) \in \mathbb{U} \text{ such that } \mathbf{u}_j \succeq \mathbf{u}_i \text{ and } \mathbf{v}_j \preceq \mathbf{v}_i\}$.

By setting $q = 2$, a 12-round key-dependent integral distinguisher is identified using Algorithm 4, presented as Distinguisher 7.

Distinguisher 7 (Key-dependent Integral Distinguisher Based on Linear Combinations of Ciphertext Bits for 12-round SKINNY-64-64). *Activating the last 60 plaintext bits yields that the following linear combinations of ciphertext bits are balanced: $b_{16} \oplus b_{32} \oplus b_{48}$ (dependent on k_0, k_1), $b_{24} \oplus b_{40} \oplus b_{56}$ (dependent on k_{16}, k_{17}), and $b_{28} \oplus b_{44} \oplus b_{60}$ (dependent on k_{20}, k_{21}).*

6.2 Probabilistic Integral Property Based on Nonlinear Combinations

Table 12 shows an 7-round integral-neural distinguisher for SKINNY-64-64, referred to as $\mathcal{IN}\mathcal{D}_4$.

Table 12. The integral-neural distinguisher $\mathcal{IN}\mathcal{D}_4$ on SKINNY-64-64

Distinguisher	Round	Data Format	Data	$\mathcal{A}$ -set	$\mathcal{B}$ -set	Acc	TPR	TNR	Ref.
$\mathcal{IN}\mathcal{D}_4$	7	DS	2^4	15th	6th	76.4%	93.5%	59.4%	Table 6

Since the division sequence $\mathbb{DS}$ used to train $\mathcal{IND}_4$ consists of only 16 bits, Algorithm 2 can be directly applied to derive its CNF, without requiring a bit-sensitivity test or retraining, to analyze the distinguisher's internal behavior. For the integral properties considered in this paper, determining the rule of a variable with n bits in CNF requires 2^{n-1} clauses. However, the rules extracted from $\mathcal{IND}_4$ contain only 2^{n-3} clauses, which is insufficient. Moreover, the CNF representation of $\mathcal{IND}_4$ contains 490 clauses in total. For clarity, only those clauses with minimal variable count are listed below:

$$\begin{aligned} & (b_6^1 \vee b_6^2 \vee b_6^8 \vee \neg b_6^9) \wedge (b_6^1 \vee \neg b_6^2 \vee b_6^8 \vee b_6^9) \\ & \wedge (b_6^1 \vee b_6^4 \vee b_6^8 \vee \neg b_6^{12}) \wedge (\neg b_6^1 \vee b_6^4 \vee b_6^8 \vee b_6^{12}) \\ & \wedge (b_6^2 \vee b_6^8 \vee \neg b_6^9 \vee b_6^{12}) \wedge (\neg b_6^2 \vee b_6^8 \vee b_6^9 \vee b_6^{12}) \end{aligned}$$

We then examine the variables involved in these clauses and evaluate the corresponding combinations of ciphertext bits to determine whether they exhibit balanced properties:

$$b_6^1 \oplus b_6^2 \oplus b_6^8 \oplus b_6^9, \quad b_6^1 \oplus b_6^4 \oplus b_6^8 \oplus b_6^{12}, \quad b_6^2 \oplus b_6^8 \oplus b_6^9 \oplus b_6^{12}.$$

For clarity, each term b_i^u is further expressed as a nonlinear combination of ciphertext bits. The corresponding Boolean expressions are given below:

$$b_{24} \oplus b_{26} \oplus b_{27} \oplus b_{24}b_{27}, \quad b_{24} \oplus b_{25} \oplus b_{27} \oplus b_{24}b_{25}, \quad b_{24} \oplus b_{26} \oplus b_{24}b_{27} \oplus b_{24}b_{25}.$$

We perform one round of backward monomial expansion and check each resulting monomial for its integral property. Among the candidates, only $b_{24} \oplus b_{25} \oplus b_{27} \oplus b_{24}b_{25}$ can be balanced by controlling the key values, as it includes key-dependent terms. In particular, when k_{28} and k_{29} are fixed to 0, the nonlinear combination of ciphertext bits becomes balanced with a probability of about 0.625. In contrast, $b_{24} \oplus b_{26} \oplus b_{27} \oplus b_{24}b_{27}$ and $b_{24} \oplus b_{26} \oplus b_{24}b_{27} \oplus b_{24}b_{25}$ cannot, since their unknown monomials are key-independent.

$$\begin{aligned} & b_{24} \oplus b_{25} \oplus b_{27} \oplus b_{24}b_{25} \\ & = 1 \oplus \underbrace{k_{31} \oplus s_{10} \oplus k_{28}k_{29} \oplus s_{11}k_{29} \oplus s_9k_{29} \oplus s_8k_{29} \oplus s_{10}k_{28} \oplus s_9k_{28} \oplus s_8k_{28}}_{\text{balanced}} \\ & \oplus \underbrace{s_8s_9k_{29} \oplus s_9s_{10}k_{28}}_{\text{unknown, key-dependent}} \end{aligned} \quad (4)$$

Using Algorithm 4, all nonlinear combinations of ciphertext bits for the 6th ciphertext cell were enumerated, yielding a total of 2^{2^4} possibilities. Based on this analysis, a key-dependent integral distinguisher involving nonlinear combinations of ciphertext bits is proposed, as summarized below.

Distinguisher 8 (Key-dependent Integral Distinguisher Based on Non-linear Combinations of Ciphertext Bits for 7-round SKINNY-64-64). *If the 15th plaintext cell is activated, the following nonlinear combinations of ciphertext bits are balanced:*

$$b_{24} \oplus b_{26} \oplus b_{24}b_{27} \text{ (dependent on } k_{28}, k_{31}), \quad b_{24} \oplus b_{25} \oplus b_{27} \oplus b_{24}b_{25} \text{ (dependent on } k_{28}, k_{29}).$$

Finally, we derive an 11-round key-dependent integral distinguisher based on a nonlinear combination of ciphertext bits against SKINNY-64-64, as shown in Distinguisher 9.

Distinguisher 9 (Key-dependent Integral Distinguisher Based on Non-linear Combination of Ciphertext Bits on 11-round SKINNY-64-64). *If the last 60 plaintext bits are activated, then the probability that the following nonlinear combinations of ciphertext bits are balanced:*

$$b_{24} \oplus b_{26} \oplus b_{24}b_{27} (\text{dependent on } k_{20}, k_{23}), b_{24} \oplus b_{25} \oplus b_{27} \oplus b_{24}b_{25} (\text{dependent on } k_{20}, k_{21}).$$

The integral properties described in Distinguishers 9 correspond to the integral property defined in Definition 4, where the function g denotes a nonlinear mapping. Hebborn’s upper bound on integral distinguishers for non-standard SKINNY was derived under the assumption of linear combinations of ciphertext bits. Our findings show that this bound also holds when nonlinear combinations are taken into account, thereby reinforcing the tightness of Hebborn’s result. This outcome may indicate that no 12-round integral distinguisher exists for SKINNY, or alternatively, that the prohibitively large search space of nonlinear combinations prevents exhaustive exploration.

7 Conclusion

This paper explores the application of neural networks in integral cryptanalysis and presents several advancements. By comparing neural network results with those of classical methods, we find that existing automated search models are inaccurate. Using an improved automated search model, we extend the length of the integral distinguishers for SKINNY. In addition to discovering longer integral distinguishers, we also improve key-recovery attacks. Specifically, the short-round distinguisher learned by the neural network enables a longer-round key-recovery attack on SKINNY through a combination of backward decryption and forward encryption. These results demonstrate that artificial intelligence techniques can offer valuable support in advancing cryptanalysis. Unfortunately, identifying integral distinguishers requires an exhaustive enumeration of all possible combination functions. Exploring more efficient search strategies remains an important direction for future research.

Acknowledgments

The authors would like to thank anonymous reviewers for their insightful and helpful comments. Special thanks to Kai Hu for helpful technical discussions and to Tianyu Zhang for suggestions on the paper title. This research is supported by the National Key R&D Program of China (Grant No. 2024YFB4504900), the National Cryptologic Science Foundation of China (Grant No. 2025NCSF02016, 2025NCSF01012), the National Natural Science Foundation of China (Grants

No. 62422214, 62472421, 62172410, 62572369), the Youth Innovation Promotion Association of Chinese Academy of Sciences, the International Partnership Program of Chinese Academy of Sciences (Grant No. 205GJHZ2024005FN), the Ministry of Education in Singapore under Grant RG102/24. We used language tools to improve readability; all technical content and claims were verified by the authors.

A Improved Key Recovery Attack in the Single Tweakey Setting

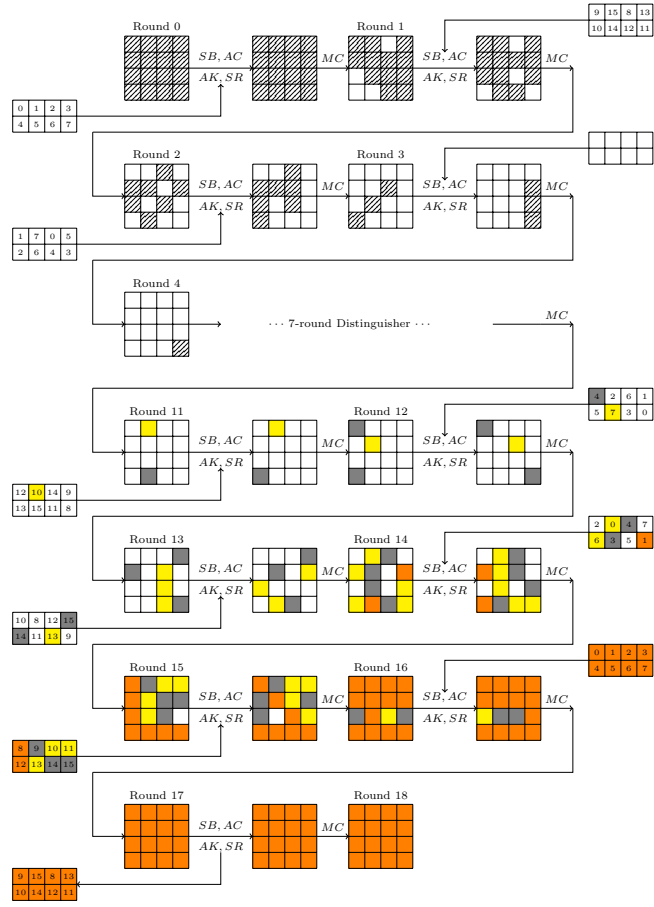


Fig. 5. 18-round key recovery attack for SKINNY- $n-2n$ in single tweakey setting

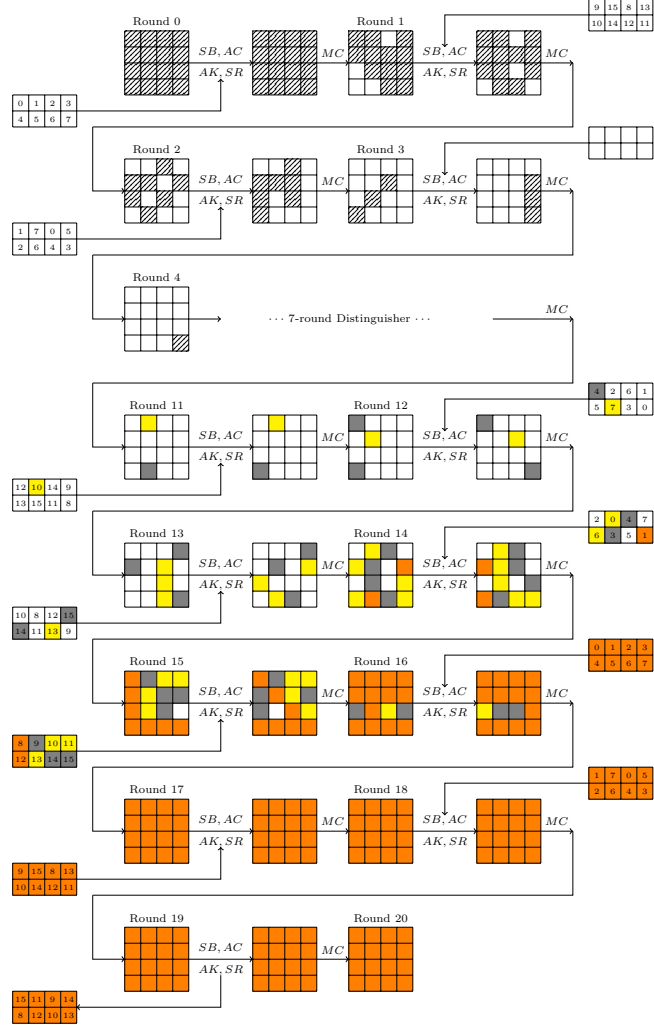


Fig. 6. 20-round key recovery attack for SKINNY- $n-3n$ in single tweak setting

B Key Recovery Attack in the Chosen Tweak Setting

Under the chosen-tweak setting, a 9-round key-independent integral distinguisher is identified when the 15th plaintext cell is activated and the 8th key byte is set to the value obtained by applying the `SubCells` and `AddConstant` to the second state byte after one round of encryption.

Distinguisher 10 (Chosen Tweak Integral Distinguisher Based on Linear Combination of Ciphertext Bits on 9-round SKINNY- n - n). *For 9-round SKINNY- n - n , when the 15th plaintext cell is activated, the following relation always holds:*

$$\bigoplus_{x_1 \in \mathbb{X}_1} \pi_u(x_1) \oplus \bigoplus_{x_2 \in \mathbb{X}_2} \pi_u(x_2) = 0,$$

for all u with $wt(u) = 1$ and ciphertext cell pairs $(\mathbb{X}_1, \mathbb{X}_2) \in \{(3, 7), (6, 14)\}$.

The overall procedure of the key-recovery attack is illustrated in Figure 7. The target cipher E is divided into three parts: E_0 covers rounds $(0 \rightarrow \dots \rightarrow 2)$, E_1 (Distinguisher 10) covers rounds $(3 \rightarrow \dots \rightarrow 11)$, and E_2 covers rounds $(12 \rightarrow \dots \rightarrow 16)$. The strategy proceeds as follows:

1. Obtain a pair of plaintext-ciphertext (P, C) .
2. Guess 14-cell involved keys,
 - Determine the value of P in $\mathbb{Z}$ of E_0 . Encrypt the plaintext P using the guessed key to obtain the ciphertext after the two rounds. Then, traverse the 15th cell of the Second round of ciphertext to obtain a ciphertext set. Next, perform 3 rounds of decryption to recover the initial plaintext set. Finally, execute 17 rounds of encryption to obtain the final ciphertext set.
 - Determine the values in $\blacksquare$ of round 12: partially decrypt each ciphertext using the guessed involved keys. Check whether the involved ciphertext bits are balanced and obtain the candidate round keys that pass the test.

The Complexity Analysis of Key Recovery Attack for SKINNY- n - n . The overall procedure of the key-recovery attack is illustrated in Figure 7. In round 1, a total of 12 cells are activated, resulting in a data complexity of $2^{12 \times c}$. A total of 16 plaintexts and ciphertexts are used, and the entire attack process involves keys from 14 cells and encryption/decryption of 64 cells. Therefore, the time complexity is calculated as $2^c \times 2^{14 \times c} \times \frac{64}{16 \times 17} = 2^{15c - 2.087}$.

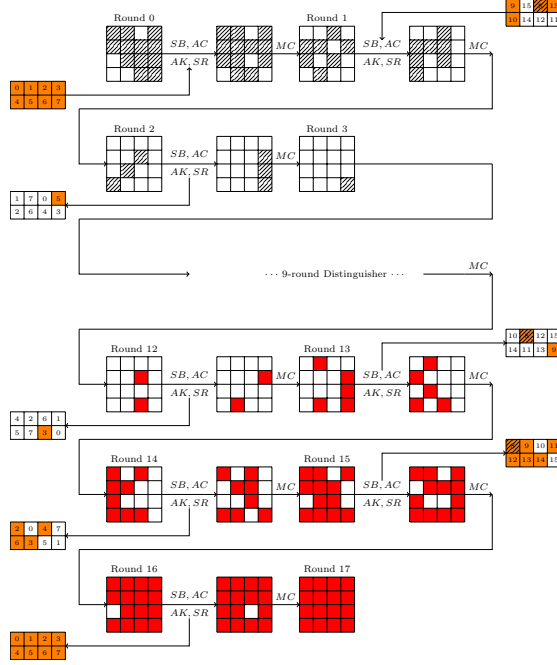


Fig. 7. 17-round chosen-tweak-key recovery attack for SKINNY- n - n

Table 13. The integral attack on SKINNY in the chosen tweak setting

Cipher	Round	Configure	Time	Data	Key Size	Reference
SKINNY-64-64	18	1+12+5	$2^{53.58}$	$2^{53.58}$	2^{60}	[12]
SKINNY-64-64	17	3+9+5	$2^{57.913}$	2^{48}	2^{60}	Appendix B
SKINNY-128-128	18	1+12+5	$2^{105.58}$	$2^{105.58}$	2^{120}	[12]
SKINNY-128-128	17	3+9+5	$2^{117.913}$	2^{96}	2^{120}	Appendix B

References

1. Bao, Z., Lu, J., Yao, Y., Zhang, L.: More insight on deep learning-aided cryptanalysis. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 436–467. Springer (2023)
2. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: CRYPTO (2). Lecture Notes in Computer Science, vol. 9815, pp. 123–153. Springer (2016)
3. Benamira, A., Gérault, D., Peyrin, T., Tan, Q.Q.: A deeper look at machine learning-based cryptanalysis. In: EUROCRYPT (1). Lecture Notes in Computer Science, vol. 12696, pp. 805–835. Springer (2021)
4. Beyne, T., Verbaauwhede, M.: Integral cryptanalysis using algebraic transition matrices. IACR Trans. Symmetric Cryptol. **2023**(4), 244–269 (2023)
5. Boura, C., Canteaut, A.: Another view of the division property. In: CRYPTO (1). Lecture Notes in Computer Science, vol. 9814, pp. 654–682. Springer (2016)
6. Chen, Y., Shen, Y., Yu, H.: Neural-aided statistical attack for cryptanalysis. Comput. J. **66**(10), 2480–2498 (2023)
7. Derbez, P., Fouque, P.A.: Increasing precision of division property. IACR Transactions on Symmetric Cryptology pp. 173–194 (2020)
8. Derbez, P., Lambin, B.: Fast milp models for division property. IACR Transactions on Symmetric Cryptology pp. 289–321 (2022)
9. Ferguson, N., Kelsey, J., Lucks, S., Schneier, B., Stay, M., Wagner, D.A., Whitling, D.: Improved cryptanalysis of rijndael. In: FSE. Lecture Notes in Computer Science, vol. 1978, pp. 213–230. Springer (2000). https://doi.org/10.1007/3-540-44706-7_15
10. Gohr, A.: Improving attacks on round-reduced speck32/64 using deep learning. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II 39. pp. 150–179. Springer (2019)
11. Gurobi Optimization, LLC: Gurobi Optimizer Reference Manual (2024), <https://www.gurobi.com>
12. Hadipour, H., Gerhalter, S., Sadeghi, S., Eichlseder, M.: Improved search for integral, impossible differential and zero-correlation attacks application to ascon, fork-skinny, skinny, mantis, PRESENT and qarmav2. IACR Trans. Symmetric Cryptol. **2024**(1), 234–325 (2024)
13. Hebborn, P., Lambin, B., Leander, G., Todo, Y.: Strong and tight security guarantees against integral distinguishers. In: ASIACRYPT (1). Lecture Notes in Computer Science, vol. 13090, pp. 362–391. Springer (2021)
14. Hu, K., Sun, S., Todo, Y., Wang, M., Wang, Q.: Massive superpoly recovery with nested monomial predictions. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 392–421. Springer (2021)
15. Hu, K., Sun, S., Wang, M., Wang, Q.: An algebraic formulation of the division property: Revisiting degree evaluations, cube attacks, and key-independent sums. In: ASIACRYPT (1). Lecture Notes in Computer Science, vol. 12491, pp. 446–476. Springer (2020)
16. Knudsen, L.R., Wagner, D.A.: Integral cryptanalysis. In: FSE. Lecture Notes in Computer Science, vol. 2365, pp. 112–127. Springer (2002)
17. Lambin, B., Derbez, P., Fouque, P.: Linearly equivalent s-boxes and the division property. Des. Codes Cryptogr. **88**(10), 2207–2231 (2020)

18. Sasaki, Y., Wang, L.: Meet-in-the-middle technique for integral attacks against feistel ciphers. In: SAC. Lecture Notes in Computer Science, vol. 7707, pp. 234–251 (2012). https://doi.org/10.1007/978-3-642-35999-6_16
19. Todo, Y.: Structural evaluation by generalized integral property. In: EUROCRYPT (1). Lecture Notes in Computer Science, vol. 9056, pp. 287–314. Springer (2015)
20. Todo, Y.: Integral cryptanalysis on full MISTY1. J. Cryptol. **30**(3), 920–959 (2017)
21. Todo, Y., Morii, M.: Bit-based division property and application to simon family. In: Peyrin, T. (ed.) Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers. pp. 357–377. Lecture Notes in Computer Science (2016)
22. Wang, S., Hu, B., Guan, J., Zhang, K., Shi, T.: Milp-aided method of searching division property using three subsets and applications. In: ASIACRYPT (3). Lecture Notes in Computer Science, vol. 11923, pp. 398–427. Springer (2019)
23. Xiang, Z., Zhang, W., Bao, Z., Lin, D.: Applying milp method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In: Advances in Cryptology–ASIACRYPT 2016: 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I 22. pp. 648–678. Springer (2016)
24. Zahednejad, B., Lyu, L.: An improved integral distinguisher scheme based on neural networks. Int. J. Intell. Syst. **37**(10), 7584–7613 (2022)
25. Zhang, W., Cao, M., Guo, J., Pasalic, E.: Improved security evaluation of SPN block ciphers and its applications in the single-key attack on SKINNY. IACR Trans. Symmetric Cryptol. **2019**(4), 171–191 (2019)